Analyzing the Development Lifecycle of Vichan

Tristan Lopez, Noah Cash, Jonathan Lange, Ar-Raniry Ar-Rasyid

Objective

Vichan is a free, open-source program for running imageboard websites. Its code has been public on GitHub since 2010, and the project officially ended in 2026. We want to use the project's change history to answer two questions:

- As the project got older, did the developers stop adding new features and spend most of their time on security and bug fixes?
- How much of the work was caused by outside problems, such as security holes, compared with the team choosing to clean up or reorganize its own code?

Motivation

Many open-source projects are run by volunteers and eventually stop. Vichan's full history, from its start to its end, is public, so we can see how the work changed over the project's whole life. Doing this by hand would mean reading thousands of changes one by one, which is slow and inconsistent. Sorting the changes automatically makes the question answerable.

Data

Source: the Vichan GitHub repository (https://github.com/vichan-devel/vichan).

What it is: every saved change (a "commit") made to the code. Each commit records who made it, when, a short message describing it, and the exact lines of code that were added or removed.

How we get it: repository mining. We download a full copy of the repository with Git, and a Python script pulls the history out into spreadsheet (CSV) files.

Amount: about 9,600 commits, which touch files about 25,000 times and contain about 65,000 separate blocks of changed code. Each changed block of code is one record, so we expect to process about 65,000 records.

Expected challenges:

- **Vague messages:** many commit messages just say "fix" or "update," so we will also look at which files and lines changed.
- **Same person, different names:** one developer used at least six different names and emails, so these need to be merged.
- **Wrong labels:** automatic sorting can make mistakes, so we need to check its results.

Approach

- **Collect:** write a Python script that downloads the history and saves it to CSV files.
- **Clean:** remove "merge" commits that only combine other work, put all dates in one time zone, and combine the different names that belong to the same person.
- **Sort:** label each change as a *Feature, Bug fix, Security fix, Code cleanup,* or *Documentation* change. We will first use keywords (for example, "XSS" or "vulnerability" means Security), then use an AI tool to label anything the keywords cannot.
- **Check:** label a small random sample by hand and compare it with the automatic labels to see how accurate they are.
- **Analyze:** count each type of change per year and graph it, so we can see whether security and bug-fix work grew as the project aged.

Tools: Git, Python, pandas, an AI tool for labeling, and Matplotlib for graphs.

Team Responsibilities

- **Jonathan Lange:** writes the script that collects the data.
- **Tristan Lopez:** cleans the data.
- **Ar-Raniry Ar-Rasyid:** builds the automatic labeling and checks its accuracy.
- **Noah Cash:** makes the graphs and writes the final report.

Timeline and Milestones

Week of	Milestone
Oct 26	Data collected into CSV files
Nov 2	Data cleaned
Nov 9	Labeling tested on a small sample
Nov 16	All data labeled and accuracy checked
Nov 23	Graphs and analysis done
Nov 30	Final report finished

Expected Outcome

A data analysis with graphs showing how Vichan's work changed over its life. It will show whether the project moved from building new features toward mostly security and bug fixes, and how much of its work was caused by outside problems.